C++ 学习档案

常见问题与学习记录

> **这份档案记录我学过的主题、遇到的问题、我的理解和接下来要巩固的内容。**
> 我可以在每次学习后继续追加，不要求一次记完所有细节。

一、数组与基础算法

1.数组长度与输入检查

> 输入数组长度后，先判断长度是否在允许范围内，再分配数组、输入元素或访问下标。长度无效时应重新输入或结束流程，不能继续使用无效长度。

写循环检查时，先确认变量已经初始化；再检查循环条件是否能让程序在输入无效时继续、输入有效时退出。

2.数组倒序

> 常见做法是从数组两端向中间交换元素：`arr[i]` 与 `arr[n - 1 - i]`。只需要交换前一半元素。

```
for (int i = 0; i < n / 2; ++i)
{
    int temp = arr[i];
    arr[i] = arr[n - 1 - i];
    arr[n - 1 - i] = temp;
}
```

容易出错的地方

- `int* arr` 形参只表示首元素地址，函数不会自动知道数组长度，所以通常还要传入 `n`。
- 不要在主函数里为了打印而把唯一的首地址指针不断 `arr++`，否则后续函数拿到的就不是原数组起点。
- 打印循环应使用真实长度 `n`，不要把长度固定写成 5。
- 用 `new[]` 创建的数组，使用完成后要用 `delete[]` 释放。

3.冒泡排序

> 通过反复比较相邻元素，在顺序不符合要求时交换。每一趟会把一个较大（或较小）的元素推到未排序区间的边界。

复习时，我要能说明：比较哪两个元素、什么时候交换、每一趟结束后哪个位置已经确定。

4.二叉树遍历

> 我学过二叉树遍历顺序这个主题。复习时可分别整理前序、中序、后序和层序遍历，并用一棵小树写出访问顺序。

二、递归与迭代

1.递归的组成

> **递归关系**：把当前问题化成规模更小的同类问题。
> **终止条件**：问题缩小到可以直接给出答案时停止调用。

以求和为例：

```
sum(n) = n + sum(n - 1)
sum(1) = 1
```

2.递归的执行顺序

> 调用时先一层层向更小的问题展开；遇到终止条件后，再从最深一层开始逐层返回结果。

```
sum(4)
= 4 + sum(3)
= 4 + 3 + sum(2)
= 4 + 3 + 2 + sum(1)
= 4 + 3 + 2 + 1
```

我当时的推理

> 我把递归理解为把问题不断拆成更小的问题，并尝试追踪 `sum(4)` 调用 `sum(3)`、再到更深层调用的过程。我也注意到上一层需要等待下一层返回，直到遇到一个确定的结果，再逐层把结果带回来。

这个思路抓住了"向下调用"和"逐层返回"两个阶段。复习时还要准确说出终止条件，并区分每一层正在等待的表达式。

3.递归与迭代

> 递归通过函数调用表达重复的子问题；迭代通常通过循环和变量更新重复执行步骤。遇到新题时，我要能分别指出状态如何变化、何时停止、结果如何得到。

三、指针、数组指针与引用

1.普通指针

```
int a = 10;
int* p = &a;
```

> `p` 保存 `a` 的地址，`*p` 表示通过这个地址访问到的整数。

2.指针与数组

> 数组名在多数表达式中会转换为首元素地址；但数组本身和指针变量不是同一种东西。数组长度不会随着地址传入普通指针形参。

3.二维数组中的数组指针

```
int arr[3][5];
int (*p)[5] = arr;
int* q = *p;
```

> p 指向一行含 5 个 int 的数组。*p 是这一行数组；在这个表达式中，数组会转换为首元素地址，因此可用于初始化 int* q。

我当时的疑问

> 我当时把普通指针的经验套到了数组指针上，觉得 p 是地址，而 *p 应该是一个整数，所以不理解 int* q = *p 为什么成立。

记忆重点

写法	含义	加一时前进
int* p	指向一个 int	一个 int
int (*p)[5]	指向含 5 个 int 的一行	一整行
int* p[5]	含 5 个 int* 的数组	这不是数组指针

4.指针与引用的常见写法

写法	含义	能否通过它修改所指对象	能否改指向/重新绑定
int* p	指向 int 的指针	可以	可以改指向
const int* p	指向只读 int 的指针	不可以	可以改指向
int* const p	常量指针	可以	不可以改指向
const int* const p	指针与所指对象都只读	不可以	不可以改指向
int& r	int 的引用	可以	不能重新绑定
const int& r	只读引用	不可以通过 r 修改	不能重新绑定

> const int& p = a; 可以让 p 成为 a 的只读引用。a 仍可通过原变量名修改，但不能通过 p 修改。

判断 const 修饰谁时，从变量声明开始读类型；不要只凭"离哪个符号最近"机械判断复杂声明。

5.函数参数里的指针、const 与引用

> 我曾在成绩统计函数中看到 int*、const int* 和引用等不同参数形式，并追问它们为什么不同。

- 需要通过地址访问一组连续元素，或需要让函数修改调用者的数据时，常见选择是指针或引用。
- 只读取数据时，可以加 const，表达函数不应通过该参数修改对象。
- 选择参数形式还要看是否需要空值、是否要表达"必须绑定一个对象"、以及数据大小和接口语义。

四、动态内存、函数与返回值

1.new 与动态数组

```
int* arr = new int[n];
// 使用 arr

delete[] arr;
```

> new[] 动态创建数组并返回首元素地址；对应使用 delete[] 释放。使用前要保证 n 合法，释放后不要继续访问这块内存。

2.return 与返回值

> return 把结果交回调用处，并结束当前函数的执行。函数返回类型决定返回结果的类型；有返回值的函数应返回合适的结果，无返回值函数使用 void。

3.返回对象与生命周期

> 我曾尝试这样理解对象返回：返回的看似是一个数，背后可能有一个没有名字的对象承载结果，再用于初始化目标对象。

补充理解

- 返回基本类型的数值，与按值返回类对象不是一回事。
- 类对象按值返回时，可能涉及复制或移动；现代 C++ 还允许或要求某些情况下直接构造、消除中间对象。
- 局部对象离开其作用域时生命周期结束；理解析构顺序时，要先列出对象在哪个作用域创建，再看它何时离开作用域。
- 实际看到的构造、拷贝和析构输出，要结合代码和 C++ 标准规则分析，不能假设每次都发生相同次数的拷贝。

五、类与对象

1.构造函数

> 构造函数在创建对象时运行，用来初始化对象。复习时要能分清构造函数名与参数名，并结合具体代码判断每个参数代表什么。

2.成员函数与当前对象

> 我曾问过：成员函数 calculateV() 没有显式形参，为什么仍能返回成员变量计算出的结果？

> 调用 object.calculateV() 时，成员函数与调用它的对象关联。非静态成员函数可以访问当前对象的成员；可以把这种关联理解为通过隐含的 this 指针找到当前对象。

六、代码排错与头文件

1.数组长度校验

> 我曾写过用 while 和 if 检查数组长度的代码。检查这类代码时，我要逐步确认：变量是否已赋值、循环何时继续、有效输入时如何退出、无效输入时是否会再次询问。

2.类定义与头文件

> 我遇到过 `point`、`circle` 相关的类型重定义和未定义类型错误。

- `#pragma once` 可以防止同一个头文件在一次编译中重复包含。
- 如果一个类按值包含另一个类的对象，编译器需要在定义处看到被包含类的完整定义。
- 头文件保护和正确的包含依赖都要检查；只加 `#pragma once` 并不能解决所有类型依赖问题。

七、当前掌握情况

知识点	当前状态	下一步目标
数组倒序与长度管理	练习中	独立写出反转函数，并正确保留首地址、处理边界长度
冒泡排序	接触过	能解释每一趟的比较范围，并手写排序代码
二叉树遍历顺序	学过相关主题，细节待整理	用同一棵树列出四种遍历结果
递归调用与返回	正在形成理解	不看提示画出调用展开、终止条件和返回过程
普通指针与数组指针	需要巩固	看声明判断指向对象和 `p + 1` 的步长
`const`、指针与引用	需要巩固	能解释各写法能否改值、改指向或重新绑定
类、构造函数与 `this`	接触过	能说明对象、成员函数和当前对象之间的关系
对象返回与析构顺序	正在建立模型	用小例子追踪生命周期，并解释拷贝消除
`new[]`、数组与释放	练习中	完整处理输入校验、分配、访问和释放
编译错误与头文件	有排错经验	能区分重复定义、未定义类型和依赖顺序问题

八、待复习清单

- [ ] 用自己的话讲清 `int*`、`const int*`、`int* const`、`int&`、`const int&`。
- [ ] 画出 `int (*p)[5]` 和 `int* q` 在二维数组中的指向关系，并说明各自加一的步长。
- [ ] 独立完成数组倒序，测试长度为 1、奇数和偶数的情况。
- [ ] 手写冒泡排序，并逐趟说明哪些位置已经排好。
- [ ] 用一棵小树写出前序、中序、后序、层序遍历。
- [ ] 追踪 `sum(4)` 的递归展开与返回，并写出循环版本。
- [ ] 用代码观察构造、复制/移动、析构，并说明中间对象可能被消除的原因。
- [ ] 检查动态数组从输入长度到 `delete[]` 的完整流程。
- [ ] 练习定位头文件重复包含和不完整类型错误。

九、后续学习记录模板

```
### 日期 | 主题

**我遇到的问题**：

**我原来的理解**：
```

我现在的解释：

代码或例题：

我验证后的结果：

掌握状态：接触中 / 能解释 / 能独立完成 / 能迁移应用

还要复习的地方：

每次学完一个新主题，我就把问题、自己的推理和验证结果补充到这份档案里。